BÖLÜM 5 / MODULE 5

# Ana Şartname Şablonu & Alan Alan Derinlemesine İnceleme

## The Master Specification Template & Field Breakdown

AI Unified Process use case şablonunun anatomisi: Genel Bakış (Overview), Önkoşullar ile Tetikleyicilerin ayrımı, 3-9 adımlı Ana Başarı Senaryosu, Başarı ve Başarısızlık Sonkoşulları (Postconditions / Guarantees), İş Kuralları (BR-XXX) ve örnek 'Borrow Book' master şartnamesinin adli analizi.

Anatomy of the AIUP specification template: Overview metadata, the strict boundary between Preconditions and Triggers, the 3-to-9 step Main Success Scenario, transactional Postconditions (Success vs Failure guarantees), Business Rules (BR-XXX), and forensic deconstruction of the canonical 'Borrow Book' spec.

25

BÖLÜM: ANA ŞARTNAME ŞABLONU & ALAN ALAN DERINLEMESINE İNCELEME | MODULE: THE MASTER SPECIFICATION TEMPLATE & FIELD BREAKDOWN

# AIUP Use Case Şablonu: Makinenin ve İnsanın Kusursuz Anladığı Format

The AIUP Use Case Template: Anatomy of a Machine-Readable Document

## ⚡ Akış, Varlık & Süreç Şeması

## ⚡ Process & Artifact Schematic

```
AI UNIFIED PROCESS MASTER USE CASE ŞABLONU

# Use Case: [Use Case Name]
## Overview
- Use Case ID: UC-XXX  |  Status: Draft / Approved / Implemented
- Primary Actor / Secondary Actors
- Goal: Tek cümlelik gözlemlenebilir hedef ve gerekçesi
- Trigger: Süreci başlatan tetikleyici olay (Durum değil, geçiş!)
- Requirements: [FR-XXX, NFR-XXX, C-XXX](../requirements.md)

## Preconditions
- [Sistemin zaten doğru olduğunu varsaydığı kapı şartları]

## Main Success Scenario
1. [Etken çatılı, tek özneli, numaralı adımlar (3 - 9 adım)]

## Alternative Flows
### A1: [Hata veya İstisna Durumu] (step N)

## Postconditions (Success & Failure Garantileri)
## Business Rules (BR-XXX)
```

## 💡 Aptala Anlatım (ELI5 Metaforu)

## 💡 ELI5 Metaphor

### 🇹🇷 TÜRKÇE (ELI5)

Bir pasaport başvuru formunu düşünün. Dünyanın her yerinde pasaport formu aynı standart kutucuklara sahiptir: Ad, Soyad, Doğum Yeri, Anne Adı. Eğer herkes kendi kafasına göre mektup şeklinde başvuru yazsaydı memurlar çıldırırdı, pasaportlar basılamazdı! AIUP şablonu da böyledir: Her use case aynı tek tip üniformayı giyer. İnsan gözü aradığı şeyi saniyesinde bulur, yapay zekâ ise metni okurken asla bocalamadan her alanı ilgili kod katmanına yerleştirir!

### 🇬🇧 ENGLISH (ELI5)

Consider an international passport application form: standardized boxes for Name, Date of Birth, Nationality, and Photo. If citizens submitted free-form handwritten essays, border control would collapse into chaos. Uniformity pays a double dividend: humans skim familiar structures in seconds, while AI generators parse labeled fields deterministically without guessing where the business logic begins.

### 🔍 MAANG & AIUP Seviyesi Teknik Derinlik

#### 🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Şablon Bütünlüğü ve Cockburn Şablonunun Evrimi: Cockburn 2001'de 'Töreni (ceremony) anlaşmazlığın maliyetine göre ayarlayın' demişti. Aynı odada çalışan küçük bir ekip günlük dilde use case yazabilirdi; ancak dağıtık ekipler 'fully dressed' (tam teşekküllü) formata muhtaçtı. AI çağında yapay zekâ, insan geliştiriciden katbekat daha uzaktır: Ne aynı koridoru paylaşır ne de kahve içer! Bu nedenle AIUP'ta her şartname istisnasız tam teşekküllü şablonla yazılır. Ancak bu şablon Cockburn'ün eski ağır şablonundan daha sadedir: 1) Kapsam (Scope) ve İrtifa (Level) satırları kaldırılmıştır; çünkü AIUP'ta kapsam daima Sistem Kapsamı, irtifa ise daima Deniz Seviyesidir (sabit kurallar tekrar edilmez). 2) Teknoloji ve Veri Varyasyonları kaldırılmıştır; çünkü teknoloji koda aittir. 3) Frekans ve Öncelik gereksinim kataloğuna (requirements.md) taşınmıştır. Geriye kalan alanlar saf, damıtılmış ve mekanik birer kontrattır.

### 🔍 MAANG & AIUP Technical Deep-Dive

#### 🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Structural Uniformity and the Modernization of Cockburn's Template: Cockburn advised calibrating documentation ceremony against the cost of misunderstanding: colocated engineers could survive on informal bullet points, whereas distributed enterprises required fully dressed forms. In generative development, the implementer (the LLM) is maximally distant. It possesses no shared culture and cannot ask for clarification. Consequently, every AIUP specification adopts a fully dressed structure. Yet the template is radically streamlined compared to classical RUP: 1) Scope and Altitude headers are omitted because they are invariant constants (System Scope, Sea Level). 2) Technology Variations are purged because technical mechanisms belong to stack plugins. 3) Prioritization and SLAs are delegated to the requirements catalog. The surviving schema is an unadulterated behavioral contract.

### ⚡ Modern Java 25 & Clean Architecture Uygulaması

### ⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: AIUP Şablonunun Domain Mimarisine Birebir İzdüşümü (Clean Ports)
package com.maang.aiup.template;

public class UseCaseTemplateStructure {
    // 1. Overview Metadata: Hedef ve Niyet
    public record Metadata(String id, String name, String goal, String trigger) {}

    // 2. Preconditions: Route Guard veya Güvenlik Ön Kontrolü
    public record Preconditions(boolean isAuthenticated, boolean isAccountActive) {
        public void verify() {
            if (!isAuthenticated || !isAccountActive) {
                throw new SecurityException("Önkoşul ihlali: Kullanıcı oturum açmamış veya hesap aktif değil!");
            }
        }
    }

    // 3. Postconditions: Başarı ve Başarısızlık Çıktı Garantileri
    public record ExecutionOutcome(boolean success, String payload, String auditLog) {}

    void main() {
        var meta = new Metadata("UC-002", "Borrow Book", "Member borrows a book", "User requests borrow");
        var pre = new Preconditions(true, true);
        pre.verify();
        System.out.println("Şablon doğrulaması başarılı: " + meta.name());
    }
}
```

### ⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

#### 🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Şablondaki başlıkların isimlerini kafanıza göre değiştirmeyin! 'Overview' yerine 'Genel Bilgiler', 'Main Success Scenario' yerine 'Adımlar' yazarsanız, downstream çalışan otomasyon ve /spec-review araçları şartnameyi parse edemez ve hata verir.

### ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

#### 🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Do not alter the markdown section headers arbitrarily. Renaming 'Main Success Scenario' to 'Steps' breaks the regex parsers in `/spec-review` and halts the automated build pipeline.

### 🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Tek tip standart şablon, insanların hızlı göz gezdirmesini ve AI'ın hatasız kod üretmesini sağlar.
- Gereksiz alanlar (Teknoloji, Kapsam) atılmıştır; sadece saf davranışsal kontrat korunmuştur.
- Başlıklar ve alan isimleri otomasyon araçları tarafından denetlenen katı arayüzlerdir.

### 🎯 Key Architectural Takeaways

- A uniform template accelerates human skimming and guarantees deterministic machine parsing.
- Obsolete fields are discarded; only the pure behavioral and transactional contract remains.
- Markdown headings are strict interfaces audited by automated quality tooling.

26

BÖLÜM: ANA ŞARTNAME ŞABLONU & ALAN ALAN DERINLEMESINE İNCELEME | MODULE: THE MASTER SPECIFICATION TEMPLATE & FIELD BREAKDOWN

# Önkoşul vs Tetikleyici (Preconditions vs Triggers): Asla Akışta Yeniden Doğrulanmayan Bekçiler

Preconditions vs Triggers: The Gatekeepers That AI Must Never Re-Validate

## ⚡ Akış, Varlık & Süreç Şeması

## ⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────┐
│              TETİKLEYİCİ (TRIGGER) VS ÖNKOŞUL (PRECONDITION)              │
└──────────────────────────────────────────────────────────┘

 TETİKLEYİCİ (TRIGGER):
 - Bir DURUM değil, bir GEÇİŞTİR (Transition from Waiting to Running)!
 - "Üye kitabı seçer ve 'Ödünç Al' butonuna tıklar." (Olay anı!)
 - Sistemin uyku halinden uyanmasını sağlayan kıvılcımdır.

 ÖNKOŞUL (PRECONDITION):
 - Senaryo başlamadan önce ZATEN DOĞRU olduğu garanti edilen durumdur.
 - "Üye oturum açmıştır" & "Üyenin hesabı aktiftir."
 - ⚠ SİSTEM BUNU ASLA SENARYO ADIMLARINDA YENİDEN KONTROL ETMEZ!
 - Kodda bir if-else değil; bir ROTA KORUYUCUSU (Route Guard / Filter) olur!
```

## 💡 Aptala Anlatım (ELI5 Metaforu)

## 💡 ELI5 Metaphor

### 🇹🇷 TÜRKÇE (ELI5)

Bir uçağa binmeyi düşünün: 1) Önkoşul: Pasaportunun olması ve güvenlik kontrolünden geçmiş olmandır (Sen daha uçağın kapısına gelmeden çok önce zaten tamamlanmıştır). 2) Tetikleyici: Hostesin 'Uçağa biniş başladı, lütfen biletinizi okutun' anonsu yapmasıdır (Hareketi başlatan an!). Uçağın içine girdikten sonra koltuğuna otururken hostes sana 'Dur bir daha pasaportunu göster, güvenlikten geçtin mi kontrol edeyim' demez (Önkoşul akış içinde tekrar doğrulanmaz!).

### 🇬🇧 ENGLISH (ELI5)

Boarding a commercial flight: 1) Precondition: Possessing a valid passport and having cleared airport security. This state was established long before reaching the boarding gate. 2) Trigger: The gate agent announcing 'Group A may now board.' (The catalyst transitioning you from waiting to boarding). Once you step down the jetway, flight attendants do not halt you in the aisle to re-screen your luggage for liquids. A precondition is never re-validated inside the execution flow.

### 🔍 MAANG & AIUP Seviyesi Teknik Derinlik

#### 🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Önkoşul Semantiği ve AI Kod Üretimindeki Önemi: Martinelli ve Cockburn önkoşul ile tetikleyici arasındaki farkı şöyle netleştirir: 1) Tetikleyici (Trigger): 'Bekleme' durumundan 'Çalışma' durumuna geçiştir. Kullanıcının bir isteği, zamana bağlı bir alarm veya harici sistemden gelen bir web-hook mesajıdır. Zaten doğru olan bir durum (ör: 'Üye kitap ister') tetikleyici olamaz; tetikleyici olayı anlatmalıdır. 2) Önkoşul (Precondition): Sistemin daha use case başlamadan önce kesinlikle doğru olduğunu garanti ettiği durumdur. Bu kuralın iki altın testi vardır: a) Sistem bunu zorlayabilir (enforce) mi? 'Üye kitabı geri getirmek niyetindedir' bir önkoşul olamaz çünkü sistem niyeti bilemez. b) Bu önkoşul daha önce başka bir use case tarafından kuruldu mu? 'Üye oturum açmıştır' önkoşulu, daha önce çalışmış olan 'UC-001 Log In' use case'i tarafından kurulmuştur. Eğer hiçbir use case kurmadıysa, o bir önkoşul değil; senaryonun 1. adımıdır! 3) AI Kuralı: Önkoşul, use case akışının içinde bir if-else kontrolü olarak YAZILMAZ; HTTP filtrelerinde veya Spring Controller `\@PreAuthorize` katmanında çözülür.

### 🔍 MAANG & AIUP Technical Deep-Dive

#### 🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Precondition and Trigger Mechanics in Generated Architectures: 1) Trigger: The catalyst event that transitions the use case from idle to active. It must describe an occurrence (an actor action, an inbound webhook, or a temporal scheduler tick), never a persistent state. 2) Preconditions: The state guarantees enforced by the system prior to execution. Cockburn established two definitive tests: First, the condition must be programmatically enforceable (an actor's emotional intent fails). Second, it must have been established by an upstream use case (e.g. 'Member is signed in' was established by `UC-001 Log In`). If no prior use case established it, it is a runtime validation for Step 1, not a precondition. 3) Architectural Translation: Preconditions compile into ingress boundary assertions or route guards. Crucially, they are never re-checked inside the scenario steps; asserting preconditions within the happy path violates the additive flow model.

### ⚡ Modern Java 25 & Clean Architecture Uygulaması

### ⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Önkoşulun (Guard) Akıştan Ayrılması (Spring Security / Gateway Modeli)
package com.maang.aiup.preconditions;

public class PreconditionEnforcementDemo {
    public record UserSession(String userId, boolean active) {}

    // ÖNKOŞUL KORUYUCUSU (Route Guard / Interceptor):
    // Use Case başlamadan önce çalışır; akışın içinde if-else kirliliği yapmaz!
    public static class PreconditionGuard {
        public static void ensurePreconditions(UserSession session) {
            if (session == null || !session.active()) {
                throw new SecurityException("Önkoşul İhlali: Oturum geçersiz veya hesap kilitli!");
            }
        }
    }

    // USE CASE: Önkoşulun sağlandığını bilir ve sadece asıl hedefe odaklanır
    public static class BorrowBookFlow {
        public void execute(UserSession session, String bookId) {
            PreconditionGuard.ensurePreconditions(session); // Kapı kontrolü
            // Akış içinde artık 'Oturum açık mı?' diye if-else yapılmaz!
            System.out.println("Ana akış çalışıyor: Kitap ödünç verildi -> " + bookId);
        }
    }

    void main() {
        var validSession = new UserSession("USR-99", true);
        new BorrowBookFlow().execute(validSession, "B-12");
    }
}
```

### ⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

#### 🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Senaryo adımlarının 1. maddesine 'Sistem kullanıcının oturum açtığını kontrol eder' YAZMAYIN! Eğer oturum açmak bir önkoşulsa zaten Önkoşullar bölümüne yazılmıştır. Senaryonun içinde tekrarlamak AI'ın gereksiz çift kontroller üretmesine yol açar.

### ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

#### 🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never write 'Step 1: System verifies user is logged in'. If authentication is a precondition, it is already guaranteed. Duplicating preconditions inside scenario steps causes AI generators to produce redundant defensive boilerplate.

### 🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Tetikleyici sistemi uyandıran olaydır (eylem); Önkoşul ise zaten var olan durumdur.
- Önkoşul sistem tarafından denetlenebilir olmalı ve daha önceki bir use case tarafından kurulmuş olmalıdır.
- Önkoşullar kodda rota koruyucusu (route guard) olur; senaryo adımlarında tekrar kontrol edilmez.

### 🎯 Key Architectural Takeaways

- A trigger is the event waking the system; a precondition is an already established state.
- Preconditions must be enforceable and established by an upstream use case.
- Preconditions translate into route guards; they are never re-checked within scenario steps.

27

BÖLÜM: ANA ŞARTNAME ŞABLONU & ALAN ALAN DERINLEMESINE İNCELEME | MODULE: THE MASTER SPECIFICATION TEMPLATE & FIELD BREAKDOWN

# Ana Başarı Senaryosu (Happy Path): 3 ila 9 Adımlı Altın Ritim ve Denge

## Main Success Scenario: The 3-to-9 Step Golden Rhythm

### ⚡ Akış, Varlık & Süreç Şeması

### ⚡ Process & Artifact Schematic

```
                ANA BAŞARI SENARYOSUNUN 3-9 ADIMLI ALTIN RİTMİ

  1. ADIM: Aktör talebini ve verilerini gönderir. (Etkileşim)
  2. ADIM: Sistem varlığı ve seçenekleri gösterir. (Etkileşim)
  3. ADIM: Aktör onaylar / işlemi talep eder. (Etkileşim)
  4. ADIM: Sistem 1. kuralı doğrular (BR-001). (Doğrulama)
  5. ADIM: Sistem 2. kuralı doğrular (BR-002). (Doğrulama)
  6. ADIM: Sistem durumu kaydeder ve stoğu düşer. (Durum Değişimi)
  7. ADIM: Sistem kullanıcıya onay ve makbuz gösterir. (Hedef Teslim Edildi!)

  ⚠ KURAL:
  - 3 adımdan azsa: Muhtemelen doğrulama ve durum değişimi adımları unutulmuştur!
  - 9 adımdan çoksa: Araya buton tıklamaları ve UI mekanikleri sızmıştır; birleştirin!
```

### 💡 Aptala Anlatım (ELI5 Metaforu)

### 💡 ELI5 Metaphor

#### 🇹🇷 TÜRKÇE (ELI5)

Mükemmel bir fıkra veya masal 3 ila 9 cümleden oluşur: Giriş, gelişme ve mutlu son. Eğer masalı 2 cümlede bitirirsen ('Keloğlan yola çıktı, kral oldu') kimse bir şey anlamaz (Çok kısa!). Eğer 25 cümle yapıp Keloğlan'ın her adımda ayağını nasıl kaldırdığını anlatırsan herkes uyuyakalır (Çok uzun!). Use case'in ana senaryosu da 3 ila 9 adımdır: Ne eksik ne fazla; hedefe dümdüz giden en temiz yoldur!

#### 🇬🇧 ENGLISH (ELI5)

A classic fairy tale spans 3 to 9 narrative beats: setup, rising action, and resolution. A 2-sentence story ('The prince rode forth and rescued the kingdom') is too abrupt to convey meaning. A 30-sentence story micromanaging how the prince buttoned his left boot puts the audience to sleep. The main success scenario requires 3 to 9 steps: lean, purposeful, and marching straight toward the goal.

### 🔍 MAANG & AIUP Seviyesi Teknik Derinlik

#### 🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Ana Başarı Senaryosunun (Main Success Scenario) Mühendislik Dinamikleri: Cockburn 20 yılı aşkın tecrübesinde 11 adımdan uzun olup da kısaltıldığında daha iyi hale gelmeyen TEK BİR use case senaryosu bile görmediğini belirtir. 1) 3-9 Adım Kuralı: İnsan beyninin çalışma belleği (working memory) Miller Kanununa göre 7±2 birim bilgiyi tutabilir. Bir paydaş veya mühendis 9 adımdan uzun bir senaryoyu kafasında canlandıramaz; doğrulama zafiyetleri başlar. 2) Adımların Şekli: Senaryoda ASLA if-else veya dallanma olamaz. Her adım MUTLAKA başarılı olur. 'Sistem şifre doğru mu bakar' denmez; 'Sistem şifrenin doğruluğunu onaylar' denir. Başarısızlıklar alternatif akışlara fırlatılır. 3) Adım Birleştirme (Collapsing): Birbiri ardına gelen mikro hareketler ('Kullanıcı adı girer', 'Kullanıcı soyadı girer', 'Kullanıcı tıklar') tek bir adıma indirgenir: '1. Kullanıcı kişisel bilgilerini girer.' Bu kural arayüz mekaniklerinin şartnameyi kirletmesini engeller.

### 🔍 MAANG & AIUP Technical Deep-Dive

#### 🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Mechanics of the Main Success Scenario: Cockburn observed that he had never seen a scenario longer than 11 steps that did not dramatically improve when shortened. 1) The 3-to-9 Step Heuristic: Rooted in cognitive science (Miller's $7 pm 2$ working memory limit). Both stakeholders validating the workflow and engineers auditing the generated architecture must hold the sequence in mind simultaneously. If a scenario has fewer than 3 steps, state mutations and validations were omitted. If it exceeds 9 steps, UI mechanical clicks have invariably infiltrated the text. 2) The Uncompromising Happy Path: The main scenario contains zero conditional branching. Every step succeeds unconditionally. Failures are decoupled into named alternative flows. 3) Step Aggregation: Unify fragmented micro-interactions ('Enters first name', 'Enters last name', 'Clicks Next') into an intent-driven bundle: 'User enters personal information.'

### ⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: 3-9 Adımlı Ana Senaryoyu Yürüten Temiz Orkestrasyon Metodu
package com.maang.aiup.scenario;

public class MainScenarioOrchestrator {
    public record BorrowCommand(String memberId, String bookId) {}
    public record BorrowReceipt(String transactionId, String dueDate) {}

    // Ana Başarı Senaryosunun 3-9 adımlı temiz icrası:
    public BorrowReceipt executeHappyPath(BorrowCommand cmd) {
        // Step 4: System validates copy availability (BR-004)
        validateAvailability(cmd.bookId());

        // Step 5: System validates loan limit (BR-005)
        validateLoanLimit(cmd.memberId());

        // Step 6: System records loan
        String txnId = persistLoanRecord(cmd.memberId(), cmd.bookId());

        // Step 7: System decrements copy count
        decrementInventory(cmd.bookId());

        // Step 8: System confirms and delivers goal
        return new BorrowReceipt(txnId, "2026-11-17");
    }

    private void validateAvailability(String b) {}
    private void validateLoanLimit(String m) {}
    private String persistLoanRecord(String m, String b) { return "TXN-101"; }
    private void decrementInventory(String b) {}

    void main() {
        var receipt = new MainScenarioOrchestrator().executeHappyPath(new BorrowCommand("M1", "B9"));
        System.out.println("✅ Ana senaryo başarıyla icra edildi: " + receipt);
    }
}
```

### ⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

#### 🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Ana senaryo adımlarında ‘Eğer stok yoksa…’ veya ‘Kullanıcı dilerse…’ gibi koşul cümleleri (if-statements) KESİNLİKLE YASAKTIR! Koşullu her durum Alternatif Akışlar (Alternative Flows) bölümüne aittir.

#### 🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never introduce ‘If the item is out of stock…’ inside the main success scenario. The main scenario asserts absolute success. All deviations and conditional forks belong exclusively in the Alternative Flows section.

### 🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Ana senaryo 3 ila 9 adımdan oluşur; dallanma içermez, her adım başarıyla tamamlanır.

• 9 adımdan uzun senaryolarda ekran detayları sızmıştır; adımları niyet bazında birleştirin.

• Son adım mutlaka kullanıcının hedefine ulaştığını ve değerin teslim edildiğini gösterir.

### 🎯 Key Architectural Takeaways

• The main scenario has 3 to 9 steps, contains zero branches, and succeeds unconditionally.

• Scenarios exceeding 9 steps suffer from UI mechanical leakage; collapse them into intent.

• The final step must unambiguously deliver the user's primary goal.

28

BÖLÜM: ANA ŞARTNAME ŞABLONU & ALAN ALAN DERINLEMESINE İNCELEME | MODULE: THE MASTER SPECIFICATION TEMPLATE & FIELD BREAKDOWN

# Sonkoşullar Kontrattır: Başarı Garantileri vs Asgari Başarısızlık Garantileri

Postconditions as Contracts: Success Guarantees vs Failure Minimums

## ⚡ Akış, Varlık & Süreç Şeması

## ⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────┐
│          SONKOŞULLAR (POSTCONDITIONS): İKİ TARAFLI GARANTİ          │
└──────────────────────────────────────────────────────────┘

BAŞARI SONKOŞULLARI (SUCCESS POSTCONDITIONS / SUCCESS GUARANTEES):
- Hedef tamamlandığında dünyanın aldığı YENİ DURUM:
  * "Üye ile kitap arasında 28 gün vadeli bir ödünç kaydı mevcuttur."
  * "Kitabın mevcut kopya sayısı 1 azaltılmıştır."
  * "Ödünç kaydı üyenin aktif listesinde görüntülenir."

BAŞARISIZLIK SONKOŞULLARI (FAILURE POSTCONDITIONS / MINIMAL GUARANTEES):
- Hangi hata olursa olsun (A1, A2, A3) sistemin verdiği ASGARİ TAAHHÜT:
  * "Hiçbir kısmi kayıt veritabanına YAZILMAMIŞTIR (Rollback garantisi!)."
  * "Kitabın mevcut kopya sayısı DEĞİŞMEMİŞTİR."
  * "Başarısızlık denemesi denetim loguna (audit log) işlenmiştir."

🎯 AI İÇİN ÖNEMİ: Sonkoşullar veritabanı @Transactional sınırını ve testlerin
                  assert (doğrulama) bloklarını doğrudan belirler!
```

## 💡 Aptala Anlatım (ELI5 Metaforu)

## 💡 ELI5 Metaphor

### 🇹🇷 TÜRKÇE (ELI5)

Bir banka ATM'sinden para çekmeyi düşünün: 1) Başarı Sonkoşulu: Cebinde 1000 TL nakit para vardır, banka hesabından 1000 TL düşmüştür (Hedef tamam!). 2) Başarısızlık Sonkoşulu: ATM tam para sayarken elektrik kesilirse sistemin sana verdiği asgari sözdür: 'Merak etme! Para vermediysem hesabından da 1 kuruş bile DÜŞMEDİM, kartını geri verdim ve log tuttum!' (Kısmi işlem yok!). Sonkoşul, sistemin ne olursa olsun arkasında çöp bırakmayacağının garantisidir.

### 🇬🇧 ENGLISH (ELI5)

Withdrawing money from an ATM: 1) Success Postcondition: You hold $100 cash in hand, and your account balance is debited by exactly $100. 2) Failure Postcondition: If the power cuts out while gears are turning, the bank provides an ironclad minimum guarantee: 'No partial withdrawal occurred, your bank balance is untouched, your card is returned, and an audit incident is logged.' Failure postconditions enforce transactional atomicity.

### 🔍 MAANG & AIUP Seviyesi Teknik Derinlik

#### 🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Sonkoşullar ve İşlemsel Bütünlük (Transactional Contract): Cockburn'ün en güçlü kavramlarından biri olan 'Guarantees' (Garantiler), AIUP şablonunda 'Success Postconditions' ve 'Failure Postconditions' olarak ikiye ayrılmıştır. 1) Başarı Sonkoşulları: Sistemin mutlu yolda yarattığı tüm kalıcı durum değişikliklerini listeler. 2) Başarısızlık Sonkoşulları (Minimal Guarantees): Sistem başarısız olduğunda neyin ASLA bozulmayacağını taahhüt eder. 'Hiçbir kısmi veri kaydedilmez' garantisi, AI'ın veritabanı işlemlerini `\@Transactional` bloğu içine almasını zorunlu kılar. 'Deneme denetim günlüğüne kaydedilir' garantisi ise AI'ın güvenlik loglarını üretmesini sağlar. 3) Testlerin Tohumu: Her iki sonkoşul bölümü de boş bırakılamaz! Çünkü üretilen otomatik testlerin `assert` blokları doğrudan bu sonkoşullardan türetilir: `assertEquals(oldBalance, newBalance)` başarısızlık sonkoşulunun doğrudan koddaki yansımasıdır. Cockburn'ün altın taktiği: Senaryoyu yazmadan ÖNCE sonkoşulları yazın ve 'Başarılı bir akışın sonunda paydaşı ne mutsuz ederdi?' diye sorun; çıkan cevaplar unuttuğunuz doğrulamaları yakalamanızı sağlar.

### 🔍 MAANG & AIUP Technical Deep-Dive

#### 🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Postconditions as Transactional Contracts: Cockburn's concept of 'Guarantees' maps directly to the dual postconditions of the AIUP template: 1) Success Postconditions: Explicitly describe the state of the universe when the goal is fulfilled: records created, inventories adjusted, notifications queued. 2) Failure Postconditions (Minimal Guarantees): Express what remains true even when disaster strikes. They are not descriptions of the failure; they are the baseline invariants the system promises under catastrophic exit (e.g. 'No partial loan is persisted', 'Pre-use-case state is restored', 'The failed attempt is recorded in the security log'). 3) Generative Significance: For an AI agent, failure postconditions dictate database transaction boundaries (`\@Transactional(rollbackFor = Exception.class)`). Furthermore, postconditions seed the assertion blocks of generated test suites. Cockburn's heuristic: draft postconditions before writing the steps, asking 'What would make stakeholders furious after an apparent success?'

### ⚡ Modern Java 25 & Clean Architecture Uygulaması

### ⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Başarı ve Başarısızlık Sonkoşullarını Garanti Eden Transactional Kod
package com.maang.aiup.postconditions;

public class TransactionalContractService {
    public record Account(String id, double balance) {}

    // Başarısızlık Sonkoşulu: Hata anında bakiye ASLA değişmez (Rollback garantisi!)
    public static Account transferMoney(Account from, Account to, double amount) {
        double originalBalance = from.balance();
        try {
            if (from.balance() < amount) {
                throw new IllegalStateException("Yetersiz bakiye!");
            }
            // Başarı Sonkoşulu: Para düşüldü
            return new Account(from.id(), from.balance() - amount);
        } catch (Exception e) {
            // Başarısızlık Garantisi Assert Edilir:
            assert from.balance() == originalBalance : "KRİTİK HATA: Bakiye bozuldu!";
            System.out.println("🛡️ Failure Postcondition devrede: Bakiye korundu, log yazıldı.");
            throw e;
        }
    }

    void main() {
        var acc = new Account("ACC-1", 500.0);
        try {
            transferMoney(acc, new Account("ACC-2", 0), 1000.0);
        } catch (Exception _) {
            System.out.println("İşlem beklendiği gibi güvenle iptal edildi.");
        }
    }
}
```

### ⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

### ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

#### 🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Başarısızlık sonkoşullarına 'Hata mesajı gösterilir' YAZMAYIN! Hata mesajı bir sonkoşul değil, alternatif akışın bir adımıdır. Sonkoşul sistemin kalıcı durumunu anlatır: 'Hiçbir veri kaydedilmedi ve hesap durumu değişmedi.'

#### 🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Do not write 'Error message is displayed' as a failure postcondition. Error notifications are transient UI steps within an alternative flow. A failure postcondition defines persistent state invariants: 'No partial loan was created and inventory remains unchanged.'

### 🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Başarı sonkoşulları hedefe ulaşıldığında dünyanın yeni kalıcı durumunu belirtir.
- Başarısızlık sonkoşulları hata anında verilen asgari garantidir (Örn: Kısmi veri yazılmaz).
- Her iki sonkoşul bölümü de zorunludur; otomatik testlerin assert blokları buradan beslenir.

### 🎯 Key Architectural Takeaways

- Success postconditions define the persistent state of the world when the goal is achieved.
- Failure postconditions define minimal guarantees (e.g. no partial state is stored).
- Both postcondition sections are mandatory; they directly seed the assertion blocks of tests.

29

BÖLÜM: ANA ŞARTNAME ŞABLONU & ALAN ALAN DERINLEMESINE İNCELEME | MODULE: THE MASTER SPECIFICATION TEMPLATE & FIELD BREAKDOWN

# İş Kuralları (BR-XXX): Yerel Numaralandırma, Nitelikli Çağrı & DRY İlkesi

Business Rules (BR-XXX): Local Numbering, Cross-Spec Referencing & DRY

### ⚡ Akış, Varlık & Süreç Şeması

### ⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────────┐
│              İŞ KURALLARI (BUSINESS RULES) REFERANS VE KAPSAM MODELİ      │
└──────────────────────────────────────────────────────────────┘

USE CASE 1 (docs/use_cases/UC-001-search-catalog.md):
## Business Rules
### BR-001: Minimum Search Characters
Arama terimi en az 3 karakter olmalıdır.

USE CASE 2 (docs/use_cases/UC-002-borrow-book.md):
## Main Success Scenario
4. System validates that the search term is valid (UC-001 BR-001). ◄── [ Nitelikli Referans! ]
5. System validates member has no overdue loans (BR-006).          ◄── [ Yerel Referans! ]

## Business Rules
### BR-006: Overdue Block
Gecikmiş borcu olan üyeler borç kapanana kadar yeni kitap alamaz.

🚫 KURAL: UC-001'deki kural metnini UC-002'ye ASLA kopyalama! Link ver!
```

### 💡 Aptala Anlatım (ELI5 Metaforu)

### 💡 ELI5 Metaphor

**🇹🇷 TÜRKÇE (ELI5)**

Bir kanun kitabını düşünün: Trafik Kanunu'nda 'Kırmızı ışıkta geçmek yasaktır (Madde 48)' yazar. Belediye otobüsleri yönetmeliğinde bu kural tekrar uzun uzun yazılmaz; 'Otobüs şoförleri Trafik Kanunu Madde 48'e uymak zorundadır' diye tek satır referans verilir. Eğer kuralı her yönetmeliğe baştan yazarsan, yarın kırmızı ışık cezası değiştiğinde bazı kitapçıklarda eski ceza kalır ve ortalık karışır! Kuralı bir kere yaz, her yerden maddesiyle (kimliğiyle) çağır!

**🇬🇧 ENGLISH (ELI5)**

In statutory law, the penal code defines traffic violations once (e.g. Article 48: Running Red Lights). Municipal bus transit regulations do not duplicate the full legal text of Article 48; they state: 'Bus operators are subject to Penal Code Article 48.' If rules are copy-pasted across fifty municipal pamphlets, an amendment to the primary statute creates immediate legal contradictions across stale copies. Define the rule once; cite it by identifier everywhere.

### 🔍 MAANG & AIUP Seviyesi Teknik Derinlik

#### 🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

İş Kuralları (Business Rules) Mimarisi ve DRY Kuralı: Cockburn, kullanım senaryoları ile iş kuralları arasındaki derin ilişkiyi şu sözle özetler: 'Alternatif akışlar ve kurallar, sistemin en ilginç ve en değerli gereksinimlerinin yaşadığı yerdir.' Martinelli bu ilişkiyi katı bir mimariye oturtur: 1) Ayrı Bölüm (Dedicated Section): İş kuralları senaryo adımlarının içine satır içi (inline) gömülmez; şartnamenin sonundaki `## Business Rules` başlığı altında toplanır. Bu sayede 3 farklı akış aynı kurala atıfta bulunabilir. 2) Yerel Numaralandırma (Local Numbering): BR numaraları her use case'in kendi içinde yereldir (`BR-001`, `BR-002`...). Bu kural merkezi bir excel tablosundaki numara çakışması krizini çözer. 3) Nitelikli Çapraz Çağrı (Qualified Cross-Spec Citation): Başka bir use case'in kuralı çağrılacaksa mutlaka use case ID'si ile nitelenir: `(UC-001 BR-002)`. Başına use case kodu konmayan çıplak bir `(BR-005)` sadece o dosyanın içinde aranır. 4) Kopyalama Yasağı: Kural metnini kopyalayıp başka dökümana yapıştırmak `/spec-review` tarafından derhal yakalanır ve engellenir.

### 🔍 MAANG & AIUP Technical Deep-Dive

#### 🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Business Rule Architecture and Qualified Scoping: Cockburn recognized that the core business value of software concentrates within rule extensions. The AI Unified Process formalizes this via rigorous conventions: 1) Decoupled Rule Section: Rules are isolated in a dedicated `## Business Rules` section rather than embedded inline within step prose. This allows multiple steps and alternative flows to link against a singular rule definition. 2) Local Identifier Namespacing: Rule numbering is strictly local per use case specification (`UC-001` owns `BR-001..003`; `UC-002` owns `BR-001..006`). This eliminates global coordination bottlenecks across distributed squads. 3) Qualified Cross-Referencing: When a specification cites a rule declared elsewhere, it must qualify the identifier: `UC-001 BR-002`. An unqualified citation (`BR-004`) refers exclusively to the host file. 4) Absolute Prohibition of Rule Cloning: Duplicating rule prose across files triggers deterministic build failure during `/spec-review` audits.

### ⚡ Modern Java 25 & Clean Architecture Uygulaması

### ⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: İş Kurallarını Merkezi Enum/Sözlük ile İzlenebilir Kılma
package com.maang.aiup.rules;

public class BusinessRuleCatalog {
    // İş kurallarının Java tarafındaki değişmez temsili
    public enum Rule {
        UC001_BR001("UC-001 BR-001", "Arama terimi en az 3 karakter olmalıdır."),
        UC002_BR005("UC-002 BR-005", "Üye aynı anda en fazla 5 açık ödünç kitaba sahip olabilir."),
        UC002_BR006("UC-002 BR-006", "Gecikmiş borcu olan üyeler ödünç alamaz.");

        public final String id;
        public final String description;
        Rule(String id, String desc) { this.id = id; this.description = desc; }
    }

    public static void validateMemberEligibility(int openLoans, boolean hasOverdue) {
        if (openLoans >= 5) {
            throw new IllegalStateException(Rule.UC002_BR005.id + ": " +
Rule.UC002_BR005.description);
        }
        if (hasOverdue) {
            throw new IllegalStateException(Rule.UC002_BR006.id + ": " +
Rule.UC002_BR006.description);
        }
    }

    void main() {
        validateMemberEligibility(3, false);
        System.out.println("✅ Tüm iş kuralları başarıyla doğrulandı.");
    }
}
```

### ⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

#### 🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir iş kuralına 'BR-001: Hızlı olmalıdır' veya 'BR-002: Güvenli olmalıdır' gibi ölçülemeyen temenniler yazmayın! İş kuralları net politikalardır: 'Üyenin en fazla 5 açık ödüncü olabilir.'

### ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

#### 🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never frame an NFR as a business rule (e.g. 'BR-001: Must respond quickly'). Business rules state definitive enterprise policy: 'A member may hold at most 5 concurrent loans.'

### 🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- İş kuralları senaryo içine gömülmez; şartnamenin sonundaki Business Rules bölümünde listelenir.
- BR numaraları use case içinde yereldir; dışarıdan çağrılırken nitelikli yazılır (UC-001 BR-002).
- Kural metinlerini dosyalar arasında kopyalamak kesinlikle yasaktır; her kural bir kere yazılır.

### 🎯 Key Architectural Takeaways

- Business rules are declared in their own dedicated section, not embedded in step text.
- BR numbering is local per spec; cross-spec citations must be qualified (UC-001 BR-002).
- Cloning rule text across specifications is forbidden; define once and cite by identifier.

30

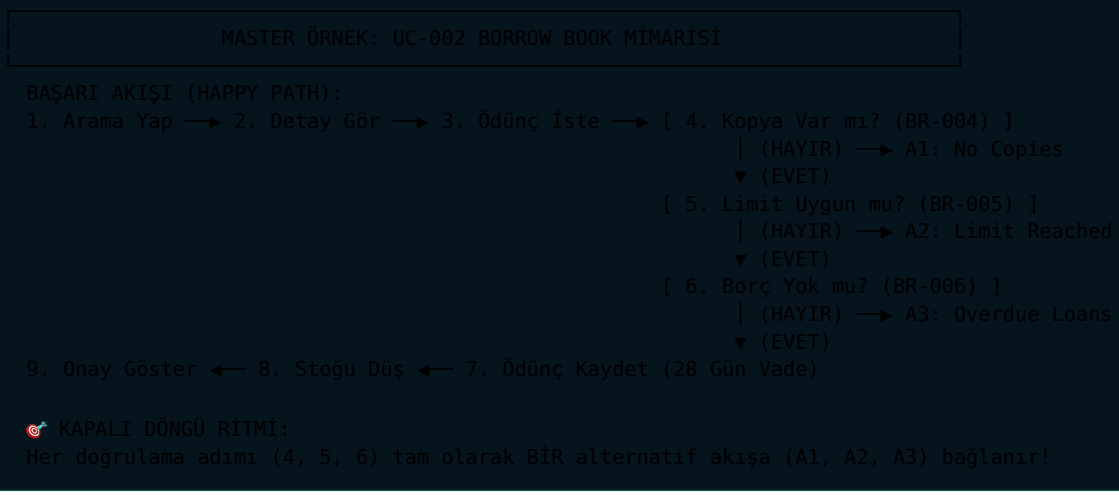
**BÖLÜM: ANA ŞARTNAME ŞABLONU & ALAN ALAN DERINLEMESINE İNCELEME | MODULE: THE MASTER SPECIFICATION TEMPLATE & FIELD BREAKDOWN**

# Baştan Sona Örnek Analizi: 'Borrow Book' (Kitap Ödünç Al) Master Şartnamesi

End-to-End Walkthrough: Deconstructing the 'Borrow Book' Master Spec

## ⚡ Akış, Varlık & Süreç Şeması

## ⚡ Process & Artifact Schematic




## 💡 Aptala Anlatım (ELI5 Metaforu)

## 💡 ELI5 Metaphor

### 🇹🇷 TÜRKÇE (ELI5)

Bu ders, bir yarış arabasının motorunu tamamen söküp parçalarını yere dizmeye benzer. 'Kitap Ödünç Al' şartnamesi, bir use case'in nasıl kusursuz yazılacağının şaheseridir: 1. adımda başlar, 9. adımda biter. 4, 5 ve 6. adımlarda 3 tane bekçi köpeği (doğrulama) oturur. Bir sorun çıkarsa köpek havlar ve seni A1, A2 veya A3 odasına yönlendirir. Sorun yoksa 28 gün vadeli kitabını alır, mutlu bir şekilde evine gidersin. Hiçbir adım havada kalmaz, hiçbir ipin ucu açıkta bırakılmaz!

### 🇬🇧 ENGLISH (ELI5)

This lesson disassembles a Formula-1 engine bolt by bolt. The canonical 'Borrow Book' specification is the gold standard of modern requirements engineering. Steps 1 to 9 march forward rhythmically. Steps 4, 5, and 6 serve as security checkpoints. If a checkpoint fails, the flow diverts cleanly to alternative branches A1, A2, or A3. If clear, the loan is committed for exactly 28 days. Every branch closes cleanly; zero dangling logic.

### 🔍 MAANG & AIUP Seviyesi Teknik Derinlik

#### 🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Borrow Book Şartnamesinin 5 Kusursuz Mimari Özelliği: Kitapta yer alan 'UC-002: Borrow Book' şartnamesi incelendiğinde her satırın arkasındaki derin mantık görülür: 1) Doğrulama ve Akış Simetrisi: 4, 5 ve 6. adımlar birer 'System validates' adımıdır. Her doğrulama adımı arkasında bir iş kuralına (BR-004, BR-005, BR-006) dayanır ve başarısızlık durumunda tam olarak bir alternatif akışa (A1, A2, A3) dallanır. 'Peki ya olmazsa?' sorusunun cevabı anında bulunur. 2) Varlık Modeli Uyumu: Metindeki 'member', 'book', 'copy', 'loan', 'reservation' kelimelerinin tamamı `docs/entity_model.md` dosyasında önceden tanımlanmıştır; use case içinde kolon tipi tartışılmaz. 3) Somut Rakamlar: '28 days' ve '5 open loans' ifadeleri mevcuttur; çünkü bunlar paydaşın onaylayacağı kesin iş rakamlarıdır. 4) Referansla Devretme (Handoff by Reference): A1 akışında kitap yoksa rezervasyon adımı için diğer use case'e link verilir: `(UC-005 Reserve Book)`. 5) Asgari Garanti: Başarısızlık sonkoşulu 'Hiçbir ödünç kaydı açılmaz ve stok değişmez' diyerek veritabanı rollback şartını belirler.

### 🔍 MAANG & AIUP Technical Deep-Dive

#### 🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Architectural Forensics of the 'Borrow Book' Specification: Deconstructing the canonical spec reveals five structural virtues: 1) Validation-to-Flow Symmetry: Steps 4 through 6 are assertions of business rules (BR-004, BR-005, BR-006). Each assertion pairs with exactly one alternative flow (A1, A2, A3) detailing systemic recovery. 2) Entity Vocabulary Alignment: The nouns 'member', 'book', 'copy', 'loan', and 'reservation' exist verbatim in `docs/entity_model.md`. The spec never defines column schemas. 3) Concrete Business Quantities: Observable constraints like '28 days' and 'at most 5 loans' are stated numerically for stakeholder sign-off. Internal date-math algorithms are omitted. 4) Handoff by Reference: In flow A1, reservation creation delegates to `(UC-005 Reserve Book)` rather than embedding child flows inline. 5) Transactional Postconditions: The failure guarantee explicitly asserts that no partial loan is persisted and inventory counts remain unchanged, dictating `\@Transactional` boundaries to the AI generator.

### ⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: 'Borrow Book' Master Şartnamesinin Birebir Java 25 Uygulaması
package com.maang.aiup.master;

import java.time.LocalDate;

public class MasterBorrowBookService {
    public record BorrowRequest(String memberId, String bookCopyId) {}
    public record LoanReceipt(String loanId, String memberId, String copyId, LocalDate dueDate) {}

    // Dışarıdan enjekte edilen bağımlılıklar (Hexagonal Outbound Ports)
    public interface BookInventoryPort {
        boolean isAvailable(String copyId); // BR-004
        void decrementStock(String copyId);
    }

    public interface MemberPort {
        int getActiveLoanCount(String memberId); // BR-005
        boolean hasOverdueLoans(String memberId); // BR-006
    }

    public interface LoanPersistencePort {
        String recordLoan(String memberId, String copyId, LocalDate dueDate);
    }

    private final BookInventoryPort inventory;
    private final MemberPort members;
    private final LoanPersistencePort loans;

    public MasterBorrowBookService(BookInventoryPort inv, MemberPort mem, LoanPersistencePort lns) {
        this.inventory = inv;
        this.members = mem;
        this.loans = lns;
    }

    public LoanReceipt borrowBook(BorrowRequest req) {
        // Step 4: System validates copy is available (BR-004 / A1)
        if (!inventory.isAvailable(req.bookCopyId())) {
            throw new IllegalStateException("A1: Kitap kopyası mevcut değil.");
        }
        // Step 5: System validates loan limit (BR-005 / A2)
        if (members.getActiveLoanCount(req.memberId()) >= 5) {
            throw new IllegalStateException("A2: Maksimum 5 ödünç limitine ulaşıldı.");
        }
        // Step 6: System validates no overdue loans (BR-006 / A3)
        if (members.hasOverdueLoans(req.memberId())) {
            throw new IllegalStateException("A3: Gecikmiş borçlar varken ödünç alınamaz.");
        }

        // Step 7: System records loan with due date 28 days from today
        LocalDate dueDate = LocalDate.now().plusDays(28);
        String loanId = loans.recordLoan(req.memberId(), req.bookCopyId(), dueDate);

        // Step 8: System decreases available copies by one
        inventory.decrementStock(req.bookCopyId());

        // Step 9: System confirms loan and delivers goal
        return new LoanReceipt(loanId, req.memberId(), req.bookCopyId(), dueDate);
    }

    void main() {
        System.out.println("Master Use Case (Borrow Book) mimari servisi hazır.");
    }
}
```

### ⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

#### 🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Borrow Book şartnamesinde '28 gün' yazmak yerine 'Sistem kütüphane kurallarına göre vade belirler' yazarsanız yapay zekayı belirsizliğe itersiniz. AI gider 14 gün yapar, testler patlar. Rakamları net yazın!

### ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

#### 🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Replacing 'due date 28 days from today' with 'system calculates appropriate due date' invites chaos. The AI might generate a 14-day default, breaking downstream acceptance assertions. State business quantities concretely.

### 🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Her 'validates' adımı arkasında bir iş kuralı taşır ve bir alternatif akışa dallanır.
- Kullanılan tüm isimler varlık modelinde mevcuttur; şema ve kolon detayları şartnamede yazılmaz.
- 28 gün ve 5 kitap gibi gözlemlenebilir sayılar şartnamede kesin olarak belirtilir.

### 🎯 Key Architectural Takeaways

- Every validation step rests on a business rule and links to an alternative flow.
- All nouns exist in the entity model; database schema details are never inlined.
- Observable quantities (28 days, 5 loans) are stated with numerical precision.